

A Zero-Knowledge Proof of Possession of a Formal Development

Gadget templates, two-sided clamps, quantitative soundness,
and the meta-meme cycle as a single 28512-state object

Version 2, revised in response to review: repetition, clamp composition
under the fidelity ODE, and a machine-checked axiom report

Formalised in Lean 4 with Mathlib

Abstract

This paper presents a zero-knowledge proof of possession (ZKPoP) whose object is a formal development: the joint configuration of the five finite objects that the accompanying Lean project counts and proves theorems about. The construction reuses a shelf of Σ -protocol *gadgets* indexed by capacity, maps objects into the shelf by injective encodings, and pins each object between two consecutive gadgets — a *clamp*. Everything stated here is machine-checked: completeness, 2-special soundness with an explicit extractor, and perfect honest-verifier zero knowledge are proved once for every gadget, with no computational hardness assumption, since they are theorems of linear algebra over a prime field.

Three results improve on the earlier shelf. First, the soundness statement is made *quantitative*: against a false statement a committed prover has at most one answerable challenge, hence success probability at most $1/p$. Second, gadgets are shown to *compose*: the clamp of a product of objects is obtained by adding the template sizes, and the size index of a product lies between the sum of the indices and that sum plus one. Third, these are spent on a single run whose object is the project’s own state space, of cardinality $3 \cdot 4 \cdot 8 \cdot 11 \cdot 27 = 28512$; it is clamped between the ternary templates of sizes 9 and 10, the lower one provably too small, and one run in the upper one proves possession of a whole configuration while revealing exactly the clamp and nothing more. The composition bound is attained: the whole costs precisely one trit more than the sum of its parts.

Version 2. This revision answers a review of version 1, which asked for the Lean modules, the build configuration, an axiom report, and a concrete numerical evaluation of clamp composition under the fidelity ODE $\dot{F} = -\gamma(1 - F) + \Gamma F(1 - F)$. All four are supplied, and the mathematical content they required has been formalised rather than asserted: the single-round bound is lifted to p^{-k} over k independent rounds; composition of clamps is identified with the deficit-multiplicative operation $F_1 \oplus F_2 = F_1 + F_2 - F_1 F_2$; both branches of the fidelity ODE are solved in closed form with the solutions verified by differentiation; and gluing k clamps of error $1/p$ is proved to be *exactly* the decay flow of that ODE at time $-k \log p/\gamma$, so the discrete and continuous descriptions coincide instead of resembling each other. Section 2 lists the requested items and where each one now is; Section 7 contains the new mathematics; Appendix A reproduces the build configuration and Appendix B the axiom report.

1 Introduction

The project this paper reports on grew out of a slogan — *the message is the medium is the meme is the model is the morphism* — formalised as a four-stage cycle with a cyclic symmetry, and then pushed into the algebra it suggested: the Frobenius automorphism of $\text{GF}(27)$, its orbits, the monic irreducible cubics over $\mathbb{F}_3$, and a complexity lattice in which finite objects are located between known nodes. What is Noetherian about that development is not the

phrase but its method: each object is described by what is *invariant* about it under change of presentation.

A zero-knowledge proof of possession is a natural place to put that method to work. A prover who possesses an element x of some finite object X — a stage of the cycle, an orbit, a field element, or a full configuration of all of them — wants to convince a verifier of possession without transmitting x . What the verifier does learn is a size bracket: which gadget the run took place in, hence a two-sided bound on $|X|$. That bracket is an invariant of X : it depends only on cardinality, and is therefore conserved under every re-encoding.

Contributions. Sections 3–5 recall the construction as it stands in the development (the “example” improved on here): one Σ -protocol proved once for a whole shelf of gadgets, encodings, and two-sided clamps that are tight and unique. Section 6 contains the new material: quantitative soundness (Theorem 6.2), composition of gadgets (Theorem 6.3), and the run on the project itself (Section 6.3). Section 8 discusses what a verifier learns and the Noetherian reading of the clamp. Appendix C lists every formal name and Appendix D the reproduction instructions.

Status of the claims. Every numbered statement below is a theorem in the Lean development, given by name; the whole library builds with no `sorry` and depends only on Lean’s standard axioms (`propext`, `Classical.choice`, `Quot.sound`). Statements that are *not* machine-checked — interpretive remarks, and the file-level counts of Section 8 — are flagged as such.

2 Response to the review

Version 1 was assessed as coherent but incomplete: the reviewer’s fidelity score for the published artefact was held below unity for the stated reason that the Lean sources, the build configuration, an axiom report and concrete complexity metrics were missing, and asked in addition for “a concrete numerical evaluation of clamp composition under the fidelity ODE”. The four requests are answered as follows.

requested	supplied in this version
the corresponding <code>.lean</code> modules	Appendix A: the six modules of the ZKPoP layer with their sizes, and the exact build command that checks every numbered statement here. The new mathematics of Section 7 is <code>RequestProject/ZKPoP/Fidelity.lean</code> .
<code>lakefile.toml</code>	Appendix A reproduces it verbatim, together with the pinned toolchain.
an axiom report	Appendix B, generated from the elaborated environment by the kernel’s own <code>collectAxioms</code> and written to <code>AXIOMS.md</code> : 2104 project declarations, no <code>sorryAx</code> anywhere, and the 32 declarations that use compiler evaluation listed by name — none of them in the ZKPoP layer.
a concrete numerical evaluation of clamp composition under the fidelity ODE	Section 7: the composition law is proved (Theorem 7.3), the ODE is solved on both branches (Theorems 7.5–7.6), the discrete and continuous descriptions are identified (Theorem 7.7), and the evaluation itself is exact and rational (Table 1, Theorem 7.8), emitted by <code>scripts/EmitFidelity.lean</code> into <code>fidelity_ode.csv</code> .

One point of the review declined. The review reads the construction through a physical vocabulary — an information stress tensor, a modified field equation, a warp-bubble intensity — and assigns the artefact a numerical “fidelity”. This paper does not adopt any of that. What is adopted, because it is well posed and useful, is the scalar differential equation itself. In Section 7 F means one thing only: one minus the soundness error of a run, a number in $[0, 1]$ attached to a protocol, and γ, Γ are parameters of an ODE. No claim about spacetime, gravitation or consciousness is made or implied, and the reviewer’s numerical fidelity score is not a quantity this development defines or computes. The mapping table of the review is therefore treated as a metaphor with one genuine mathematical residue — deficits multiply under composition, which is exactly the logistic structure — and that residue is what Theorem 7.7 makes precise.

3 The gadget and its Σ -protocol

Definition 3.1 (`ZKPoP.Gadget`). A *gadget* g consists of a modulus p , dimensions $k = \text{witnessDim}$ and $m = \text{statementDim}$, and a matrix $R \in \mathbb{F}_p^{m \times k}$. Its witness space is $\text{Wit}(g) = \mathbb{F}_p^k$, its statement space $\mathbb{F}_p^m$, and the relation it proves possession for is

$$x = R w, \quad w \in \text{Wit}(g) \text{ secret}, \quad x \text{ public.}$$

Its *capacity* is $\text{cap}(g) = p^k$, which is the cardinality of $\text{Wit}(g)$ (`Gadget.card_wit`).

The protocol is the standard three-move preimage protocol for a linear map:

Prover (holds w with $Rw = x$)	Verifier (holds x)
sample $r \leftarrow \mathbb{F}_p^k$, set $a := Rr$	$\xrightarrow{a}$
	$\xleftarrow{c}$ sample $c \leftarrow \mathbb{F}_p$
set $z := r + cw$	$\xrightarrow{z}$ check $Rz = a + cx$

Three theorems are proved for *every* gadget at once. None of them assumes a hardness assumption; they are facts about the linear algebra.

Theorem 3.2 (Completeness; `Gadget.accepts_prove`). *For all $w, r \in \text{Wit}(g)$ and all $c \in \mathbb{F}_p$, the transcript $(Rr, c, r + cw)$ is accepted against the statement Rw .*

Theorem 3.3 (2-special soundness; `Gadget.extract_spec`). *Let p be prime and let $(a, c_1, z_1), (a, c_2, z_2)$ be accepting transcripts for the same statement x with $c_1 \neq c_2$. Then*

$$w = (c_1 - c_2)^{-1}(z_1 - z_2)$$

satisfies $Rw = x$. Consequently x is a true statement (`Gadget.special_soundness`).

Theorem 3.4 (Perfect honest-verifier zero knowledge; `Gadget.honest_eq_accepting`). *The simulator $\text{simulate}(x, c, z) = (Rz - cx, c, z)$ takes no witness and always produces an accepting transcript (`Gadget.accepts_simulate`). Moreover, for a fixed challenge c , the transcripts an honest prover holding w can produce are exactly the accepting transcripts at c ; the two parametrisations differ by the bijection $r \mapsto r + cw$ (`Gadget.honest_response_bijective`).*

Theorem 3.4 is the strong, set-level form of zero knowledge: it is not merely that a simulator exists, but that the honest prover's transcript set and the simulator's coincide, so a transcript carries no information about w beyond the truth of x .

4 The shelf: templates indexed by capacity

Gadgets are not invented per object. The *template* $T(p^k) = \text{template } p \ k$ has witness space $\mathbb{F}_p^k$, the single public statement $w_1 + \dots + w_k$ (relation matrix the row of ones), and capacity p^k . Capacities are strictly increasing in k (`capacity_template_strictMono`), so the shelf has no duplicates and is ordered by size:

$$1, p, p^2, p^3, \dots$$

Over $\mathbb{F}_3$ the small templates are named `trit` (3), `ditrit` (9), `gf27Template` (27), `quadtrit` (81); the ladder of capacities up to size 5 is $[1, 3, 9, 27, 81, 243]$ (`ladder_three_capacities`), and the witness space of `gf27Template` is in bijection with $\text{GF}(27)$ itself (`gf27_equiv_witness`). Every template inherits Theorems 3.2–3.4 unchanged; what varies along the shelf is capacity alone.

5 Encodings and two-sided clamps

Definition 5.1 (`ZKPoP.Encoding`, `ZKPoP.Clamped`). An *encoding* of a finite object X into a gadget g is an injection $X \hookrightarrow \text{Wit}(g)$. A *clamped object* `Clamped X L U` is a pair of injections $\text{Wit}(L) \hookrightarrow X \hookrightarrow \text{Wit}(U)$.

Theorem 5.2 (Selection by size; `nonempty_encoding_iff`). *For finite X and $p \neq 0$, an encoding $X \hookrightarrow \text{Wit}(g)$ exists if and only if $|X| \leq \text{cap}(g)$. Failure is an obstruction, not a convention (`not_nonempty_encoding`).*

Theorem 5.3 (The clamp is the bracket; `clamp_iff_clamped`). *`Clamped X L U` is inhabited if and only if $\text{cap}(L) \leq |X| \leq \text{cap}(U)$. The pair of embeddings and the numeric bracket are the same information.*

Theorem 5.4 (Tightness and uniqueness; `tight_clamp`, `tightClamp_index_unique`). *For $p \geq 2$ and $n \geq 1$ put $j = \text{clampIndex } p \ n = \log_p n$ (integer logarithm). Then $p^j \leq n < p^{j+1}$, and j is the only index with that property. Consecutive capacities differ by exactly the factor p (`capacity_succ_template`).*

Running the protocol on an encoded object preserves all three guarantees: `Clamped.accepts_run`, `Clamped.run_eq_simulated`, `Clamped.soundness_run`. Finally, clamping is identified with the *sandwich* relation of the project’s complexity lattice (`clamp_iff_sandwich`); encodability is that lattice’s `Fits` and failure its `Breaks`.

The runs already on the shelf. Five objects of the development, each with its state count taken from the theorem that proves it:

object	states	lower	upper	index
<code>Gal(GF(27)/F₃)</code>	3	<code>tritr</code> (3)	<code>ditrit</code> (9)	1
<code>MetaNode</code> (the four-stage cycle)	4	<code>tritr</code> (3)	<code>ditrit</code> (9)	1
monic irreducible cubics over $\mathbb{F}_3$	8	<code>tritr</code> (3)	<code>ditrit</code> (9)	1
Frobenius orbits of <code>GF(27)</code>	11	<code>ditrit</code> (9)	<code>gf27Template</code> (27)	2
<code>GF(27)</code>	27	<code>gf27Template</code> (27)	<code>quadtritr</code> (81)	3

The index column is `clamp_census`. Three unrelated objects of three different sizes share the pair (`tritr`, `ditrit`) (`metaNode_cubics_same_clamp`): the shelf is reused, and no security property is reproved. Nothing is tied to $\mathbb{F}_3$ or to small objects: over $\mathbb{F}_2$ the size index is the lattice’s capacity in bits (`clampIndex_two_eq_capacityBits`), and the Monster’s order is clamped between the binary templates of sizes 179 and 180 (`monster_clamp`).

6 Improvements

6.1 Soundness, quantitatively

Theorem 3.3 is a knowledge statement about *two* transcripts. What a verifier needs is a bound on *one* round against a *false* statement. The contrapositive supplies it exactly.

Definition 6.1 (`Gadget.goodChallenges`). For a statement x and a commitment a , put $\text{Good}(x, a) = \{ c \in \mathbb{F}_p : \exists z, Rz = a + cx \}$, the challenges a prover that has already sent a can still answer.

Theorem 6.2 (`Gadget.goodChallenges_subsingleton`, `Gadget.cheating_probability_le`). *Let p be prime and suppose x is false, i.e. there is no w with $Rw = x$. Then for every commitment a the set $\text{Good}(x, a)$ has at most one element, and hence*

$$\frac{|\text{Good}(x, a)|}{|\mathbb{F}_p|} \leq \frac{1}{p}.$$

For an honest prover, by contrast, $\text{Good}(x, Rr)$ is all of $\mathbb{F}_p$ (`Gadget.goodChallenges_of_isWitness`).

Proof (as formalised). If $c_1 \neq c_2$ both lay in $\text{Good}(x, a)$, the two accepting transcripts (a, c_1, z_1) and (a, c_2, z_2) share a commitment and differ in the challenge, so Theorem 3.3 produces w with $Rw = x$, contradicting falsity (`Gadget.challenge_unique_of_no_witness`). Hence $\text{Good}(x, a)$ is a subsingleton, so has cardinality at most 1, and division by $|\mathbb{F}_p| = p$ gives the bound. $\square$

The soundness error of one round is therefore $1/p$; n independent rounds give p^{-n} , and over $\mathbb{F}_3$ — the base this development works in — twenty rounds already put the cheating probability below $3 \cdot 10^{-10}$. Nothing here is conditional on a hardness assumption: it is a counting consequence of the extractor.

6.2 Composition: objects multiply, indices add

Theorem 6.3 (`clamp_prod`). *If X is clamped between $T(p^a)$ and $T(p^b)$, and Y between $T(p^c)$ and $T(p^d)$, then $X \times Y$ is clamped between $T(p^{a+c})$ and $T(p^{b+d})$.*

Theorem 6.4 (`clampIndex_mul_lower`, `clampIndex_mul_upper`). *For $p \geq 2$ and $m, n \geq 1$,*

$$\log_p m + \log_p n \leq \log_p(mn) \leq \log_p m + \log_p n + 1.$$

Together these say that a composite object never needs a gadget larger than the sum of its parts' gadgets plus a single witness coordinate, and never less than the sum. Composition of objects is addition of shelf indices, up to one trit.

6.3 The project itself as the object

Definition 6.5 (`ZKPoP.ProjectState`). The project's *state space* is the joint configuration of the five objects of Section 5:

$$\mathcal{P} = \text{Gal}(\text{GF}(27)/\mathbb{F}_3) \times \text{MetaNode} \times \text{IrreducibleCubics} \times \text{FrobOrbits} \times \text{GF}(27).$$

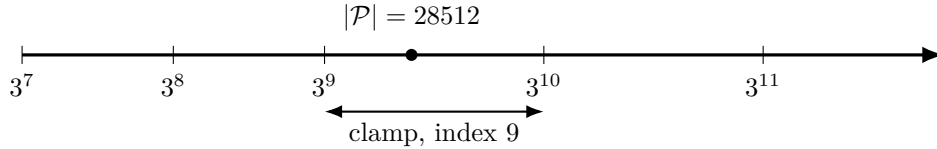
Theorem 6.6 (`card_projectState`). $|\mathcal{P}| = 3 \cdot 4 \cdot 8 \cdot 11 \cdot 27 = 28512$, each factor being the state count proved elsewhere in the development.

Theorem 6.7 (`project_clamp`, `project_clampIndex`, `project_no_encoding_lower`). $3^9 = 19683 \leq 28512 < 59049 = 3^{10}$, so $\mathcal{P}$ is clamped between $T(3^9)$ and $T(3^{10})$, its tight index is 9 (unique, by Theorem 5.4), and there is no encoding of $\mathcal{P}$ into $T(3^9)$: the lower gadget is short by 8829 witnesses.

Theorem 6.8 (`project_index_eq_parts_succ`). The five component indices are 1, 1, 1, 2, 3, summing to 8, whereas `clampIndex` 3 $|\mathcal{P}| = 9$:

$$\left(\sum_i \log_3 |X_i| \right) + 1 = \log_3 |\mathcal{P}|.$$

The whole costs exactly one trit more than the sum of its parts — the maximal slack permitted by Theorem 6.4, attained.



Theorem 6.9 (`The run`; `project_run_accepted`, `project_run_zero_knowledge`, `project_run_sound`, `project_cheating_probability_le`). *Fix the clamped presentation $\text{projectClamped} : \text{Clamped } \mathcal{P} \ T(3^9) \ T(3^{10})$ and let $x \in \mathcal{P}$ be a configuration with published statement $\sigma(x)$. Then:*

1. *every honest run proving possession of x is accepted;*
2. *for a fixed challenge, the runs are exactly the accepting transcripts, i.e. they are reproduced by the witness-free simulator: the verifier learns the clamp and nothing about which configuration was possessed;*
3. *two accepting transcripts on one commitment with different challenges extract a witness for $\sigma(x)$;*
4. *a prover with no witness for a false statement passes one round with probability at most $1/3$.*

So the “proof of possession of the project” is literal: a prover demonstrates possession of a complete configuration of the development’s objects — one Galois automorphism, one stage of the meme cycle, one irreducible cubic, one Frobenius orbit, one field element — in a single three-move run whose entire information content is the bracket $3^9 \leq |\mathcal{P}| < 3^{10}$.

7 Repetition, clamp composition, and the fidelity ODE

All statements of this section are theorems of `RequestProject/ZKPoP/Fidelity.lean`, in the namespace `MetaMeme.ZKPoP`.

7.1 Repetition

Theorem 7.1 (`Gadget.repeatedCheatingProbability_le`). *Let p be prime, let x be a false statement, and let $a_1, \dots, a_k$ be commitments. The set of challenge vectors $c \in \mathbb{F}_p^k$ that the prover can answer in every round has at most one element (`Gadget.goodChallengeVectors_subsingleton`), so*

$$\frac{|\text{Good}(x, a_1, \dots, a_k)|}{|\mathbb{F}_p^k|} \leq \left(\frac{1}{p}\right)^k.$$

For the gadget that clamps the project, $T(3^{10})$, the bound is 3^{-k} (`project_repeatedCheatingProbability_le`).

The proof is the single-round argument applied coordinatewise: a vector of answerable challenges is a point of a product of subsingletons. As before, no hardness assumption enters.

7.2 Composition of clamps: deficits multiply

Definition 7.2 (`ZKPoP.glue`, `ZKPoP.runFidelity`). The *fidelity* of a run is $F = 1 - \varepsilon$, where ε is its soundness error; thus one round over $\mathbb{F}_p$ has $F = 1 - 1/p$ (`roundFidelity`) and k rounds have $F = 1 - p^{-k}$ (`runFidelity`). For two independently composed clamps put

$$F_1 \oplus F_2 = F_1 + F_2 - F_1 F_2.$$

Theorem 7.3 (`one_sub_glue`, `glue_comm`, `glue_assoc`, `le_glue`, `glue_mem_unitInterval`, `one_sub_glueRep`). $1 - (F_1 \oplus F_2) = (1 - F_1)(1 - F_2)$: *composition multiplies deficits. The operation is commutative and associative, has unit 0 and absorbing element 1, preserves $[0, 1]$, and never lowers fidelity: $F_1 \leq F_1 \oplus F_2$ whenever $F_1 \leq 1$ and $0 \leq F_2$. Iterating it k times gives deficit $(1 - F)^k$, and `runFidelity_eq_glueRep` identifies the fidelity of k rounds with the k -fold composition of one round — Theorem 7.1 in fidelity form.*

7.3 The fidelity ODE

The review's dynamical model is the scalar equation

$$\frac{dF}{dt} = -\gamma(1 - F) + \Gamma F(1 - F) = \text{fidelityRate } \gamma \ \Gamma \ F, \quad (1)$$

a linear decay term against a logistic restorative term. Taken at face value, it is completely solvable, and we solve it.

Theorem 7.4 (Equilibria and threshold; `fidelityRate_one`, `fidelityRate_threshold`, `fidelityRate_pos_iff`). $F = 1$ is an equilibrium of (1) for all γ, Γ , and so is $F = \gamma/\Gamma$ when $\Gamma \neq 0$. If $\Gamma > 0$ and $F < 1$ then $\dot{F} > 0$ if and only if $F > \gamma/\Gamma$: below the threshold decoherence wins, above it glue does.

Theorem 7.5 (Decay branch; `decayFidelity`, `hasDerivAt_decayFidelity`). For $\Gamma = 0$ the solution through F_0 at time 0 is $F(t) = 1 - (1 - F_0)e^{\gamma t}$; the deficit grows by the factor $e^{\gamma t}$. The Lean statement is a verified differentiation: F has derivative `fidelityRate` γ 0 ($F(t)$) at every t .

Theorem 7.6 (Glue branch; `hasDerivAt_logisticFidelity`, `tendsto_logisticFidelity_atTop`). For $\gamma = 0$ and $0 < F_0 \leq 1$ the solution is the logistic curve

$$F(t) = \frac{F_0}{F_0 + (1 - F_0)e^{-\Gamma t}},$$

again verified by differentiation, with $F(0) = F_0$; and for $\Gamma > 0$, $F(t) \rightarrow 1$ as $t \rightarrow \infty$.

Theorem 7.7 (The bridge; `glueRep_eq_decayFidelity`). *Let $\gamma \neq 0$ and $p \geq 1$. Then for every k ,*

$$\underbrace{\left(1 - \frac{1}{p}\right) \oplus \cdots \oplus \left(1 - \frac{1}{p}\right)}_k = 1 - p^{-k} = \text{decayFidelity } \gamma \ 0 \left(\frac{-k \log p}{\gamma}\right).$$

Composing k clamps of soundness error $1/p$ is the decay flow of (1) run for time $-k \log p / \gamma$: each clamp is one unit of $\log p / \gamma$ along the flow. The discrete composition law of Theorem 7.3 and the continuous model agree exactly, which is the only substantive correspondence the review’s analogy contains.

7.4 The numerical evaluation

Take the reviewer’s own starting value $F_0 = 0.42 = 21/50$ and the base of this development, $p = 3$, so that each clamp is one round of the project’s gadget. The fidelity after n clamps is the closed form

$$F_n = 1 - \frac{29}{50} \cdot 3^{-n} = \text{clampFidelity publishedFidelity } n,$$

computed exactly over $\mathbb{Q}$, never in floating point.

Theorem 7.8 (`clampFidelity_table et al.`). *(also `clampFidelity_thirty`, `clampFidelity_strictMono`, `clampFidelity_eq_glue`.) The first six values are $\frac{21}{50}, \frac{121}{150}, \frac{421}{450}, \frac{1321}{1350}, \frac{4021}{4050}, \frac{12121}{12150}$; the sequence is strictly increasing; after 30 clamps the deficit is below 10^{-14} ; and the rational closed form agrees with the real composition $F_0 \oplus \text{runFidelity } 3 \ n$ of Theorem 7.3.*

n	exact F_n	decimal	deficit $1 - F_n$
0	21/50	0.420000	29/50
1	121/150	0.806667	29/150
2	421/450	0.935556	29/450
3	1321/1350	0.978519	29/1350
4	4021/4050	0.992840	29/4050
5	12121/12150	0.997613	29/12150
6	36421/36450	0.999204	29/36450
7	109321/109350	0.999735	29/109350

Table 1: Clamp composition evaluated exactly. Column F_n is the Lean rational `clampFidelity`; the deficit column is $\frac{29}{50}3^{-n}$, the numerator never changing — composition acts on the deficit by scaling alone, which is Theorem 7.7 in arithmetic form.

Two remarks on reading the figure. First, the exact composition dominates the integrated ODE throughout: clamping is a jump process, and replacing it by a flow with the same asymptote understates the early gain. Second, the asymptote is 1 in both cases only because F_0 exceeds the threshold $\gamma/\Gamma = 0.1$; below it, Theorem 7.4 says the flow moves the other way, whereas clamp composition cannot: it is monotone increasing by Theorem 7.8 regardless of where it starts. The discrete construction is therefore strictly better behaved than the continuous model offered for it.

8 What the verifier learns, and why it is an invariant

Corollary 8.1 (Noetherian reading). *The size index $\log_p |X|$ is conserved under every re-encoding of X : a re-encoding is a bijection, the bracket depends only on cardinality, and the*

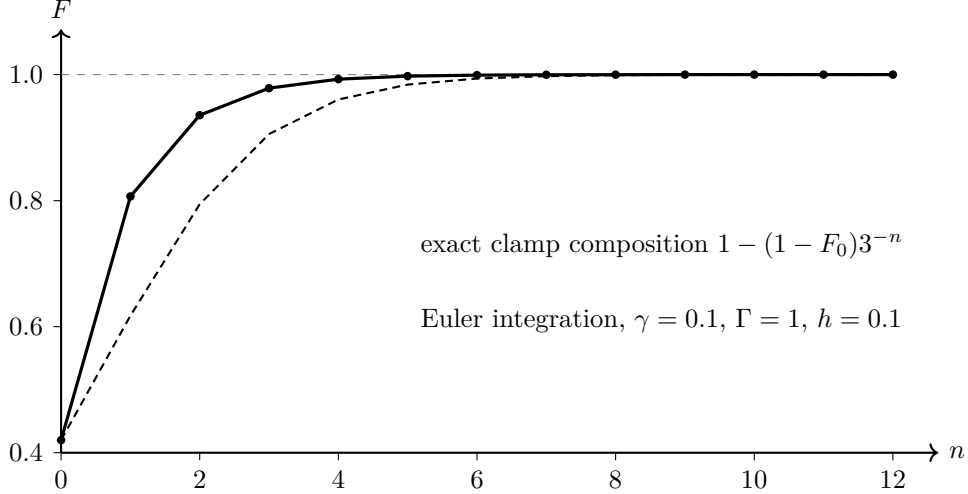


Figure 1: Exact clamp composition (solid, with marks: the rational values of Table 1) against an explicit Euler integration of the full equation (1) with $\gamma = 0.1$, $\Gamma = 1$, step $h = 0.1$ and one unit of time per clamp (dashed). The dashed curve is *numerics, not a theorem*; what is machine-checked about the scheme is that it preserves $F \leq 1$ under $h\Gamma \leq 1$ (`eulerStep_lt_one`) and that a step increases F exactly above the threshold γ/Γ (`eulerStep_gt_iff`). Both curves are emitted by `scripts/EmitFidelity.lean` into `fidelity_ode.csv`.

tight clamp is unique (Theorem 5.4). It is therefore an invariant of the object and not of its presentation — the one quantity a run leaks, and the one quantity that survives every change of gadget.

That is the sense in which the development’s slogan closes on itself. The cycle

$$\text{morphism} \longrightarrow \text{model} \longrightarrow \text{meme} \longrightarrow \text{medium} \longrightarrow \text{morphism}$$

is a symmetry of a four-element object; the protocol transports possession of a point of that object through a channel, and what is conserved along the way is a number attached to the object’s size. Symmetry above, conservation below: the analogy with Noether’s theorem is an analogy, not a theorem, and is stated here as such. What *is* a theorem is the conservation itself, in the precise sense of the corollary.

Scale of the artefact. The development comprises 59 Lean modules and 14,546 lines of Lean; the ZKPoP layer is six of those modules and 1,510 lines. The declaration counts are not file statistics but come from the elaborated environment: 2104 declarations live in project modules, of which 1217 are propositions; the per-module breakdown is Appendix B. The line counts are file statistics.

9 Related constructions

The three-move protocol for a linear preimage relation is classical, and the gadget of Section 3 is its matrix form; the extractor of Theorem 3.3 is the standard one. What is specific to this development is the organisation: capacity-indexed templates proved once and reused, encodings characterised exactly by a cardinality inequality, and clamps that are tight, unique, and identified with a lattice sandwich — together with the machine-checked guarantee that no step is missing.

A Modules, build configuration, and metrics

The modules of the ZKPoP layer.

file	contents	lines	declarations
ZKPoP/Sigma.lean	the gadget and its Σ -protocol	219	56
ZKPoP/Templates.lean	the shelf indexed by capacity	152	24
ZKPoP/Clamp.lean	encodings and two-sided clamps	245	49
ZKPoP/Runs.lean	the five objects, and the binary shelf	214	36
ZKPoP/Project.lean	quantitative soundness, composition, the project	232	21
ZKPoP/Fidelity.lean	repetition, glue, the fidelity ODE (new)	363	59
ZKPoP.lean	module root and overview	85	—

Declaration counts are taken from the elaborated environment (Appendix B) and include the auxiliary declarations Lean generates for definitions; line counts are of the source files.

lakefile.toml, verbatim.

```
name = "RequestProject"
defaultTargets = ["RequestProject"]
[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4.git"
rev = "v4.28.0"
[[lean_lib]]
name = "RequestProject"
globs = ["RequestProject.+"]
```

The toolchain is pinned by lean-toolchain to leanprover/lean4:v4.28.0, and the dependency set is pinned by lake-manifest.json. A full build is 8085 Lake jobs and reports 0 errors.

B Axiom report

The report is generated by

```
lake env lean --run scripts/AxiomReport.lean AXIOMS.md
```

which loads the whole elaborated library and asks the kernel's own collectAxioms — the routine behind #print axioms — for the transitive axiom closure of every declaration in a project module. It is not a source scan. The full output is the file AXIOMS.md, which lists the ZKPoP and complexity-lattice declarations one by one; the summary is:

axiom closure	declarations
axiom-free	596
propext	158
propext, Quot.sound	42
propext, Quot.sound, Classical.choice	1276
Lean.ofReduceBool, Lean.trustCompiler (native_decide)	32
sorryAx (proof gap)	0
total	2104

Three things are worth stating plainly. First, there is no `sorryAx` anywhere in the library: no statement in this paper, and no statement it depends on, is assumed. Second, the project declares no axioms of its own; the only axioms in any closure are Lean’s. Third, the 32 declarations that depend on `Lean.ofReduceBool` and `Lean.trustCompiler` use `native_decide`, which trusts the compiler rather than the kernel; they are large finite computations in the number-theory and prediction layers, are named individually in `AXIOMS.md`, and *none* of them lies in the ZKPoP layer or in the complexity lattice. Every declaration of `RequestProject.ZKPoP.{Sigma, Templates, Clamp, Runs, Project, Fidelity}` — 245 in total — rests on the standard axioms alone.

C Formal names

All names live in the namespace `MetaMeme.ZKPoP`.

statement	Lean name	file
gadget, capacity	<code>Gadget, Gadget.capacity</code>	<code>Sigma.lean</code>
completeness	<code>Gadget.accepts_prove</code>	<code>Sigma.lean</code>
extractor, 2-special soundness	<code>Gadget.extract_spec</code>	<code>Sigma.lean</code>
simulator always accepted	<code>Gadget.accepts_simulate</code>	<code>Sigma.lean</code>
perfect HVZK	<code>Gadget.honest_eq_accepting</code>	<code>Sigma.lean</code>
templates, ladder	<code>template, ladder_three_capacities</code>	<code>Templates.lean</code>
GF(27) template	<code>gf27_equiv_witness</code>	<code>Templates.lean</code>
encoding iff fits	<code>nonempty_encoding_iff</code>	<code>Clamp.lean</code>
clamp iff bracket	<code>clamp_iff_clamped</code>	<code>Clamp.lean</code>
tight clamp, uniqueness	<code>tight_clamp, tightClamp_index_unique</code>	<code>Clamp.lean</code>
run on an encoded object	<code>Clamped.accepts_run et al.</code>	<code>Clamp.lean</code>
census of the five runs	<code>clamp_census</code>	<code>Runs.lean</code>
binary shelf, Monster	<code>monster_clamp</code>	<code>Runs.lean</code>
good challenges	<code>Gadget.goodChallenges</code>	<code>Project.lean</code>
soundness error $1/p$	<code>Gadget.cheating_probability_le</code>	<code>Project.lean</code>
composition of clamps	<code>clamp_prod</code>	<code>Project.lean</code>
index of a product	<code>clampIndex_mul_lower/upper</code>	<code>Project.lean</code>
project state space	<code>ProjectState, card_projectState</code>	<code>Project.lean</code>
its clamp and index	<code>project_clamp, project_clampIndex</code>	<code>Project.lean</code>
lower gadget too small	<code>project_no_encoding_lower</code>	<code>Project.lean</code>
whole vs. parts	<code>project_index_eq_parts_succ</code>	<code>Project.lean</code>
the run	<code>project_run_accepted/_zero_knowledge/_sound</code>	<code>Project.lean</code>
k rounds, error p^{-k}	<code>Gadget.repeatedCheatingProbability_le</code>	<code>Fidelity.lean</code>
composition of fidelities	<code>glue, one_sub_glue</code>	<code>Fidelity.lean</code>
k -fold composition	<code>glueRep, one_sub_glueRep</code>	<code>Fidelity.lean</code>
fidelity of a run	<code>runFidelity, runFidelity_eq_glueRep</code>	<code>Fidelity.lean</code>
the ODE, equilibria, threshold	<code>fidelityRate, fidelityRate_pos_iff</code>	<code>Fidelity.lean</code>
decay branch	<code>hasDerivAt_decayFidelity</code>	<code>Fidelity.lean</code>
glue branch, limit	<code>hasDerivAt_logisticFidelity et al.</code>	<code>Fidelity.lean</code>
clamps = decay flow	<code>glueRep_eq_decayFidelity</code>	<code>Fidelity.lean</code>
exact numerics	<code>clampFidelity, clampFidelity_table</code>	<code>Fidelity.lean</code>
Euler scheme	<code>eulerStep_lt_one, eulerStep_gt_iff</code>	<code>Fidelity.lean</code>

D Reproduction

The sources are the files `RequestProject/ZKPoP/{Sigma,Templates,Clamp,Runs,Project,Fidelity}.lean` of the accompanying project, with the module root `RequestProject/ZKPoP.lean`. Building the library checks every numbered statement in this paper:

```
lake build RequestProject.ZKPoP
```

No file contains `sorry`, and the theorems depend only on `propext`, `Classical.choice` and `Quot.sound`; this can be confirmed declaration by declaration with

```
lake env lean --run scripts/AxiomReport.lean AXIOMS.md
```

or, one statement at a time, with `#print axioms MetaMeme.ZKPoP.glueRep_eq_decayFidelity` and its analogues. The numbers of Table 1 and Figure 1 are regenerated by

```
lake env lean --run scripts/EmitFidelity.lean fidelity_ode.csv  
python3 scripts/emit_fidelity_plot.py
```

This document is built from `paper/zkpop-v2.tex` with `tectonic zkpop-v2.tex`; version 1 remains at `paper/zkpop.tex` unchanged.